

Programming Project 2

● Graded

Student

Jinku Jung

Total Points

57 / 60 pts

Autograder Score

30.0 / 30.0

Passed Tests

CreditCard.java compiles with CreditCardTester.java (15/15)

isValid() returns False for bad card 3 (1/1)

isValid() returns False for bad card 6 (1/1)

isValid() returns False for bad card 4 (1/1)

isValid() returns False for bad card 5 (1/1)

getErrorCode returns 3 for bad card 3 (1/1)

getErrorCode returns 5 for bad card 5 (1/1)

getErrorCode returns correctly (1/1)

getErrorCode returns 0 for good card (1/1)

getErrorCode returns 6 for bad card 6 (1/1)

isValid() returns True for a good card (1/1)

isValid() returns False for bad card 1 (1/1)

isValid() returns False for bad card 2 (1/1)

getErrorCode returns 4 for bad card 4 (1/1)

getErrorCode returns 1 for bad card 1 (1/1)

getErrorCode returns 2 for bad card 2 (1/1)

Question 2

Autograder Adjustment

0 / 0 pts

✓ + 0 pts Autograder correct

Question 3

String Loops

10 / 10 pts

✓ + 2 pts A Correct

✓ + 2 pts B Correct

✓ + 2 pts C Correct

✓ + 2 pts D Correct

✓ + 2 pts E Correct

Question 4

Credit Card and String Loops: Style

7 / 7 pts

✓ + 2 pts Header includes UNI, name, and brief file description

✓ + 1 pt Appropriate use of comments in methods

✓ + 1 pt Descriptive variable names

✓ + 1 pt Line length under 80 characters

✓ + 2 pts Correct use of white space

Question 5

Credit Card: Design

6 / 8 pts

✓ + 4 pts Breaks checks into separate helper methods

✓ + 2 pts Instance variables are private

Many of your instance variables don't need to exist

Question 6

Credit Card: Je ne sais quois

4 / 5 pts

✓ + 1 pt Used an instance variable for error code

✓ + 1 pt Methods are shorter than 15 lines

✓ + 1 pt Each methods are not both accessor and mutator methods

✓ + 1 pt Used a loop in check 4 to sum digits

Question 7

Format and Readme

0 / 0 pts

✓ + 0 pts Good README.txt

Autograder Results

CreditCard.java compiles with CreditCardTester.java (15/15)

isValid() returns False for bad card 3 (1/1)

Testing: 4807-6052-2766

isValid() returns False for bad card 6 (1/1)

Testing: 4807-6012-1766

isValid() returns False for bad card 4 (1/1)

Testing: 4817-6052-1766

isValid() returns False for bad card 5 (1/1)

Testing: 4807-6052-1366

getErrorCode returns 3 for bad card 3 (1/1)

Testing: 4807-6052-2766

getErrorCode returns 5 for bad card 5 (1/1)

Testing: 4807-6052-1366

getErrorCode returns correctly (1/1)

Testing: 4807-6052-1766

getErrorCode returns 0 for good card (1/1)

Testing: 4807-6052-1766

getErrorCode returns 6 for bad card 6 (1/1)

Testing: 4807-6012-1766

isValid() returns True for a good card (1/1)

Testing: 4807-6052-1766

isValid() returns False for bad card 1 (1/1)

Testing: 5807-6052-1766

isValid() returns False for bad card 2 (1/1)

Testing: 4806-6052-1766

getErrorCode returns 4 for bad card 4 (1/1)

Testing: 4817-6052-1766

getErrorCode returns 1 for bad card 1 (1/1)

Testing: 5807-6052-1766

getErrorCode returns 2 for bad card 2 (1/1)

Testing: 4806-6052-1766

Submitted Files

```
1  README.txt
2  Name: Jinku Jung
3  UNI: dj2598
4  Assignment: Programming Project #2
5
6  =====
7  Big Java: Exercise 6.3, parts (a) through (e).
8  =====
9
10 a.) To find all the uppercase letters in a given string, I:
11
12     i.) Test each string element against the range of valid ASCII codes
13         for all uppercase letters, 65 to 90, inclusive.
14
15     ii.) As they are found, I concatenate them to my output string.
16
17     iii.) Last, I display the desired output string onto the screen.
18
19
20 b.) To find all the even characters in a given string, from the second
21 character onward in my for loop, I:
22
23     i.) Pluck every second character of the input string.
24
25     ii.) Then, I concatenate each character to my output string, before
26         the end of each loop cycle.
27
28     iii.) Last, I display the desired output string onto the screen.
29
30 c.) To find all the vowel characters in a given string, I:
31
32     i.) Use their ASCII interger code equivalents as a Boolean test, inside
33         an "if" conditional statement.
34
35     ii.) As they are found, each vowel is replaced by the '_'
36         (underscore character).
37
38     iii.) Last, I display the desired string onto the screen.
39
40 d.) To find the number of total vowel characters in a given string, I again:
41
42     i.) Use their ASCII integer equivalents as a Boolean test inside an
43         "if" conditional statement.
44
45     ii.) As they are found, the variable for the number of vowels found
46         is incremented by 1.
47
48     iii.) Last, I display the desired output string onto the screen.
49
```

```
50 =====
51 Create the CreditCard class *.java file.
52 =====
53
54 Here, I tried to keep my accessor and mutator methods separate all throughout
55 my CreditCard class-definition java file. Basically, for my Constructor
56 method, I:
57
58     1.) Take in the CCN as an input string.
59
60     2.) Use substring commands to pluck out only the numeric segments
61     of the said string.
62
63     3.) Concatenate those string segments (of purely numerals) to a
64     new sequentially-conjoined string.
65
66     4.) Use the Double.parseDouble() command to convert that string
67     to a double value.
68
69 The required Acessor and Mutator methods are then implemented, satisfying
70 the command requirements and parameters used in the CreditCardTester.java
71 file. Both sample CCN's checked out to be "valid" from the README.md
72 document.
73
74 CODER'S NOTE: I can't wait to use arrays, because this task could've been
75 coded in 30 minutes if they were allowed!!! --Jinku
76
```

```
1  /* Name: Jinku Jung
2   * UNI: dj2598
3   * Assignment: Programming Project #2
4   * Date: 2/10/2021
5   * Problem: Create the CreditCard class definition file in Java.
6   */
7
8  import java.lang.Math.*;
9
10 public class CreditCard {
11
12     //INSTANCE VARIABLES
13
14     private String creditCardNumber = "";
15     private String segment1Str = "";
16     private String segment2Str = "";
17     private String segment3Str = "";
18     private String CCN = "";
19     private String pureCCNStr = "";
20
21     private double pureCCN = 0;
22
23     private int errorCode = 0;
24
25
26     //CONSTRUCTOR:
27
28     // 1.) Here, I create the string-type dummy variable CCN to hold the
29     //     input parameter, creditCardNumber.
30     //
31     // 2.) Then, I break CCN into three separate strings named
32     //     segment1, segment2, and segment3.
33     //
34     //     ***** I do it this way in order to extract JUST the numbers
35     //     and OMIT the dashes inside the original credit card number
36     //     input supplied by the user. *****
37     //
38     // 3.) Later, I concatenate these very same string segments into
39     //     another string dummy variable, pureCCNStr.
40     //
41     // 4.) Finally, I parse pureCCNStr into a double-value type, to be
42     //     held from now on in the double-type pureCCN variable for
43     //     all instantiations of the CreditCard class.
44     //
45     //     ***** NOTE: I choose a double value instead of an int value
46     //                     for pureCCN because the integer value won't
47     //                     accommodate a 12-digit number of the given
48     //                     credit card number's size in base 10
49     //                     (e.g, a number such as 4XX,XXX,XXX,XXX or
```

```

50 //          400 billion+, as a 32-bit Java integer
51 //          value; talk about a stack overflow!). *****
52
53 public CreditCard(String creditCardNumber) {
54
55     CCN = creditCardNumber;
56
57     segment1Str = CCN.substring(0, 4);
58     segment2Str = CCN.substring(5, 9);
59     segment3Str = CCN.substring(10, 14);
60
61     pureCCNStr = segment1Str + segment2Str + segment3Str;
62
63     pureCCN += Double.parseDouble(pureCCNStr);
64
65 }
66
67 //ACCESSOR METHODS
68
69 // 1.) IsValid() is a Boolean function without any input parameters.
70 //    Its job is simply to return a "true" value to its caller
71 //    if and only if the e n t i r e credit card number is valid.
72
73 public Boolean isValid() {
74
75     if((getErrorCode()) == 0)
76
77     {
78         return true;
79     }
80
81     else {
82
83         return false;
84
85     }
86 }
87
88 // 2.) GetErrorCode() is an accessor method without any input
89 //    parameters. Its job is simply to return the interger value
90 //    of the variable, errorCode, to its caller function.
91
92 public int getErrorCode() {
93
94     return errorCode;
95
96 }
97
98 // 3.) GetDigit() is an accessor method that grabs the i'th numeral
99 //    of the 12-digit credit card number selected. It returns the
100 //    corresponding digit from the double-type variable, pureCCN,
101 //    using modulo arithmetic and exponential math.

```



```

102
103 public int getDigit(int i) {
104
105     int cardDigit = 0;
106
107     cardDigit = (int) ((pureCCN % Math.pow(10, (13 - i))
108         / (Math.pow(10, (13 - (i + 1))))));
109
110     return cardDigit;
111 }
112
113 //MUTATOR METHODS
114
115 // 1.) Check() is a mutator method without any input parameters.
116 // Its job is simply to run the six checksum mutator methods as
117 // validity tests of the credit card number supplied by the
118 // user. It returns no value to its caller function.
119
120 public void check() {
121
122     if(isValid()) {test1();}
123     if(isValid()) {test2();}
124     if(isValid()) {test3();}
125     if(isValid()) {test4();}
126     if(isValid()) {test5();}
127     if(isValid()) {test6();}
128
129 }
130
131 // 2.) Test1() is the first checksum mutator method for the credit
132 // card number selected. If this method finds an error,
133 // setErrorCode() is called upon, with the corresponding value
134 // of the variable, errorCode, to be set.
135
136 public void test1() {
137
138     int digit1 = getDigit(1);
139
140     if(digit1 != 4) {setErrorCode(1);}
141
142 }
143
144 // 3.) Test2() is the second checksum mutator method for the credit
145 // card number selected. If this method finds an error,
146 // setErrorCode() is called upon, with the corresponding value
147 // of the variable, errorCode, to be set.
148
149
150 public void test2() {
151
152     int digit4 = getDigit(4);
153     int digit5 = getDigit(5);

```

```
154
155     if(digit4 != (digit5 + 1)) {setErrorCode(2);}
156
157 }
158
159 // 4.) Test3() is the third checksum mutator method for the credit
160 //   card number selected. If this method finds an error,
161 //   setErrorCode() is called upon, with the corresponding value
162 //   of the variable, errorCode, to be set.
163
164 public void test3() {
165
166     int digit1 = getDigit(1);
167     int digit5 = getDigit(5);
168     int digit9 = getDigit(9);
169
170     if((digit1 * digit5 * digit9) != 24) {setErrorCode(3);}
171
172 }
173
174 // 5.) Test4() is the fourth checksum mutator method for the credit
175 //   card number selected. If this method finds an error,
176 //   setErrorCode() is called upon, with the corresponding value
177 //   of the variable, errorCode, to be set.
178
179 public void test4() {
180
181     int digitSum = 0;
182
183     for(int i = 1; i <= 12; i++) {
184
185         digitSum += getDigit(i);
186     }
187
188     if((digitSum % 4) != 0) {setErrorCode(4);}
189
190 }
191
192 //6.) Test5() is the fifth checksum mutator method for the credit
193 //   card number selected. If this method finds an error,
194 //   setErrorCode() is called upon, with the corresponding value
195 //   of the variable, errorCode, to be set.
196
197 public void test5() {
198
199     int digitSum1 = 0;
200     int digitSum2 = 0;
201
202     for (int i = 1; i <= 4; i++ ) {digitSum1 += getDigit(i);}
203     for (int j = 9; j <= 12; j++) {digitSum2 += getDigit(j);}
204
205     if ((digitSum1) != (digitSum2 - 1)) {setErrorCode(5);}
```

```
206
207     }
208
209
210 //7.) Test6() is the sixth checksum mutator method for the credit
211 //   card number selected. If this method finds an error,
212 //   setErrorCode() is called upon, with the corresponding value
213 //   of the variable, errorCode, to be set.
214
215 public void test6() {
216
217     int digit1 = getDigit(1);
218     int digit2 = getDigit(2);
219     int digit7 = getDigit(7);
220     int digit8 = getDigit(8);
221
222     if(((digit1 * 10) + digit2) +
223         ((digit7 * 10) + digit8)
224         != 100)
225         {setErrorCode(6);}
226 }
227
228 //8.) SetErrorCode() is a mutator method that takes an
229 //   integer input parameter 'i' AND increments the
230 //   variable, errorCode, by 'i'. By default, errorCode
231 //   is initialized as zero since I assume all credit
232 //   card numbers to be valid--until proven otherwise.
233
234 public void setErrorCode(int i) {
235
236     errorCode += i;
237 }
238 }
```

```
1
2  /**
3   * A tester for the CreditCard class.
4   *
5   * @author Cannon
6   */
7  import java.util.Scanner;
8
9  public class CreditCardTester {
10     public static void main(String[] args) {
11         System.out.println("Please enter a 12-digit credit card number.");
12         Scanner scanner = new Scanner(System.in);
13         String creditCardNumber = scanner.next();
14         CreditCard card = new CreditCard(creditCardNumber);
15         card.check();
16         if (card.isValid()) {
17             System.out.println("Valid");
18         }
19         else {
20             int code = card.getErrorCode();
21             System.out.println("Oops! That's an invalid number.");
22             System.out.println("The error code was: " + code);
23         }
24     }
25 }
26
27
```

```
1  /* Name: Jinku Jung
2   * UNI: dj2598
3   * Assignment: Programming Project #2
4   * Date: 2/10/2021
5   * Problem: E 6.3a from Big Java
6   */
7
8
9  // My implentation of E_6_3, part (a):
10 //
11 // 1.) Pluck all uppercase letters from the
12 //    input string value, as supplied by
13 //    the user.
14 //
15 // 2.) Then, return the resulting string as
16 //    a display output value.
17 //
18 // 3.) Finally, thank the user for running my program.
19
20 import java.util.Scanner;
21
22 public class E63a {
23
24     public static void main(String[] args) {
25
26         Scanner input = new Scanner(System.in);
27
28         System.out.println("Enter your desired input here.");
29         String testString = input.nextLine();
30         String output = "";
31
32         for (int i = 0; i < testString.length(); i++)
33         {
34             char ch = testString.charAt(i);
35             int asciiValue = (int) ch;
36             if(asciiValue >= 65 && asciiValue <= 90) {output += ch;}
37         }
38
39         String update = "Here's all the uppercase letters from your input: ";
40         System.out.println("\n" + update + output + "\n");
41         System.out.println("\n Thank you for using my program! --Jinku \n");
42     }
43 }
```

```
1  /* Name: Jinku Jung
2   * UNI: dj2598
3   * Assignment: Programming Project #2
4   * Date: 2/10/2021
5   * Problem: E 6.3b from Big Java
6   */
7
8  // My implentation of E_6_3, part (b):
9  //
10 // 1.) Pluck every 2nd character from the
11 //    user's supplied input string.
12 //
13 // *****Do this by starting with the second
14 //    charcter of the string--as you see
15 //    it as a display input value, before
16 //    you press the return key on your
17 //    keyboard.*****
18 //
19 // 2.) Then, return the resulting string of
20 //    every 2nd character from the input value
21 //    as a concatenated display output value.
22 //
23 // 3.) Finally, thank the user for running my program.
24
25 import java.util.Scanner;
26
27 public class E63b {
28
29     public static void main(String[] args) {
30
31         Scanner input = new Scanner(System.in);
32
33         System.out.println("\n Enter your desired input here. \n");
34         String testString = input.nextLine();
35
36         String update = "Picking every 2nd character of the original input: ";
37
38         System.out.println("\n" + update + "\n");
39
40         String output = "";
41
42         for (int i = 1; i < testString.length(); i += 2)
43         {
44             char ch = testString.charAt(i);
45
46
47             output += Character.toString(ch);
48
49         }
```

```
50
51     System.out.println("\n Your desired output is now: " + output);
52     System.out.println("\n Thank you for using my program! --Jinku \n");
53 }
54 }
55
56
```

```
1  /* Name: Jinku Jung
2   * UNI: dj2598
3   * Assignment: Programming Project #2
4   * Date: 2/10/2021
5   * Problem: E 6.3c from Big Java
6   */
7
8  // My implentation of E_6_3, part (c):
9  //
10 // 1.) Replace every instance of a vowel
11 //    from the input string, supplied by
12 //    the user, with the '_' underscore
13 //    character.
14 //
15 // 2.) Then, return the resulting string of
16 //    all the vowels from the input
17 //    string as a concatenated display
18 //    output string.
19 //
20 // 3.) Finally, thank the user for running my program.
21
22 import java.util.Scanner;
23
24 public class E63c {
25
26     public static void main(String[] args) {
27
28         Scanner input = new Scanner(System.in);
29
30         System.out.println("\n Enter your desired input here. \n");
31         String testString = input.nextLine();
32
33         String update = "Replacing every instance of a vowel with '_' now: ";
34         System.out.println("\n" + update + "\n");
35
36         for (int i = 0; i < testString.length(); i++)
37         {
38             char ch = testString.charAt(i);
39             int asciiValue = (int) ch;
40
41             if(asciiValue == 65 || asciiValue == 69 || asciiValue == 73
42                || asciiValue == 79 || asciiValue == 85 || asciiValue == 97
43                || asciiValue == 101 || asciiValue == 105
44                || asciiValue == 111 || asciiValue == 117)
45
46             {
47
48                 String prefix = testString.substring(0, i);
49                 String suffix = testString.substring(i + 1);
```



```
50
51     String underscore = Character.toString('_');
52
53     testString = prefix + underscore + suffix;
54
55     }
56 }
57 System.out.println("\n Your desired output is now: " + testString);
58 System.out.println("\n Thank you for using my program! --Jinku \n");
59 }
60 }
61
62
```

```
1  /* Name: Jinku Jung
2   * UNI: dj2598
3   * Assignment: Programming Project #2
4   * Date: 2/10/2021
5   * Problem: E 6.3e from Big Java
6   */
7
8  // My implentation of E_6_3, part (e):
9  //
10 // 1.) Count every instance of a vowel
11 //   from the input string supplied by
12 //   the user by iterating through
13 //   each element of the string literal.
14 //
15 // 2.) Output the visual position of the
16 //   original input string, as the user
17 //   saw it as he typed AND not as the
18 //   string element's stored position
19 //   in its corresponding string literal.
20 //
21 // *****Do this for each cycle of the for
22 //   loop whenever a vowel position is
23 //   found.*****
24 //
25 // 3.) Finally, thank the user for running my program.
26
27 import java.util.Scanner;
28
29 public class E63e {
30
31     public static void main(String[] args) {
32
33         Scanner input = new Scanner(System.in);
34
35         System.out.println("\n Enter your desired input here. \n");
36         String testString = input.nextLine();
37
38         String update = "Vowels occur at your input's following characters: ";
39         System.out.println("\n" + update + "\n");
40
41         for (int i = 0; i < testString.length(); i++)
42         {
43             char ch = testString.charAt(i);
44             int asciiValue = (int) ch;
45
46             if(asciiValue == 65 || asciiValue == 69 || asciiValue == 73 ||
47                asciiValue == 79 || asciiValue == 85 || asciiValue == 97 ||
48                asciiValue == 101 || asciiValue == 105 || asciiValue == 111 ||
49                asciiValue == 117)
```

```
50         {System.out.println("Character " + (i + 1));}  
51     }  
52     System.out.println("\n Thank you for using my program! --Jinku \n");  
53 }  
54 }  
55  
56
```

```
1  /* Name: Jinku Jung
2   * UNI: dj2598
3   * Assignment: Programming Project #2
4   * Date: 2/10/2021
5   * Problem: E 6.3d from Big Java
6   */
7
8  // My implentation of E_6_3, part (d):
9  //
10 // 1.) Count every instance of a vowel
11 //   from the input string supplied by
12 //   the user. Do this by iterating
13 //   through each element of the
14 //   supplied string literal value.
15 //
16 // 2.) Before the end of each iteration,
17 //   store the running vowel tally
18 //   using the variable, nVowels.
19 //
20 // 3.) Then, return the resulting integer
21 //   count of vowels from the input
22 //   string as a concatenated display
23 //   output value.
24 //
25 // 4.) Finally, thank the user for running my program.
26
27 import java.util.Scanner;
28
29 public class E63d {
30
31     public static void main(String[] args) {
32
33         Scanner input = new Scanner(System.in);
34
35         System.out.println("Enter your desired input here.");
36         String testString = input.nextLine();
37         String output = "The total number of vowels you have in your input is ";
38         int nVowels = 0;
39
40         for (int i = 0; i < testString.length(); i++)
41         {
42             char ch = testString.charAt(i);
43             int asciiValue = (int) ch;
44
45             if(asciiValue == 65 || asciiValue == 69 || asciiValue == 73
46                || asciiValue == 79 || asciiValue == 85 || asciiValue == 97
47                || asciiValue == 101 || asciiValue == 105
48                || asciiValue == 111 || asciiValue == 117)
```

```
50     {
51         nVowels ++;
52     }
53 }
54 System.out.println(output + nVowels + ".");
55 System.out.println("\n Thank you for using my program! --Jinku \n");
56 }
57 }
58
59
```
